

Subject 4: Structured Output Generation with Formal Grammar Constraints

1. The 80/20 Core (Hiring-Critical)

What Actually Matters (20% → 80% Impact)

Included (Production & Interview Critical):

1. **Schema-First Design with Pydantic** (35% of value)
 - Type-safe output models with validation
 - Nested schemas and complex types (unions, literals, enums)
 - Runtime validation and error handling
 - Schema evolution and backward compatibility
 - Why: Prompting alone fails 20-40% of the time; schemas enforce correctness
2. **Function Calling / Tool Use (Reliable Structured Outputs)** (25% of value)
 - OpenAI function calling, Anthropic tool use
 - Multi-function orchestration
 - Parallel tool calls for efficiency
 - Error handling and retry logic
 - Why: Most reliable method for structured outputs in production
3. **Grammar-Constrained Generation** (20% of value)
 - JSON mode vs. constrained decoding (Outlines, Guidance)
 - Performance tradeoffs (speed vs. guarantee)
 - When to use each method
 - Why: Guarantees valid outputs but at latency/compatibility cost
4. **Validation Strategies & Error Recovery** (15% of value)
 - Graceful degradation on parse failures
 - Partial extraction from malformed outputs
 - Retry with stricter prompts
 - Validation error feedback loops
 - Why: Even "guaranteed" methods fail in edge cases; must handle errors
5. **Schema Design for LLM Compatibility** (5% of value)
 - Simplicity vs. expressiveness tradeoffs
 - Naming conventions that LLMs understand

- Documentation strings as implicit prompts
- Why: Well-designed schemas = higher success rates

Explicitly Excluded (Low ROI / Interview Anti-Signals):

- **✗** Regex parsing of LLM outputs
 - *Why excluded:* Brittle, fails on formatting variations, signals amateur
- **✗** "Just prompt it to output JSON"
 - *Why excluded:* 20-40% failure rate; not production-ready
- **✗** Custom parsers for structured text
 - *Why excluded:* Reinventing the wheel; use existing tools
- **✗** XML output parsing (unless domain-specific requirement)
 - *Why excluded:* JSON dominates; XML adds complexity
- **✗** "Hope and pray" error handling
 - *Why excluded:* Production systems need deterministic error recovery

Hiring Signal Justification

What Tier 1 interviewers look for:

- Can you design Pydantic schemas that LLMs reliably populate?
- Do you understand when function calling vs. JSON mode vs. constrained decoding?
- Have you implemented validation with graceful fallbacks?
- Can you debug why an LLM fails to generate valid JSON?
- Do you version schemas and handle backward compatibility?

What signals "student not engineer":

- "I just ask the LLM to output JSON in the prompt"
 - No validation layer (trusting LLM output blindly)
 - No error recovery strategy
 - Using regex to parse JSON (instead of proper parsers)
 - Not understanding tokenization effects on structured outputs
-

2. Genius-Level Understanding

Core Mental Models

Perspective 1: Infrastructure Engineer

Structured outputs are contracts between LLM and downstream systems. Contracts must be enforced, not hoped for.

Traditional APIs:

python

```
# Type-safe, compiler-enforced
def get_user(user_id: int) -> User:
    return User(id=1, name="Alice", email="alice@example.com")

user = get_user(123)
print(user.email) # Type checker guarantees this exists
```

LLM APIs (naive approach):

python

```
# Not type-safe, runtime failure
def extract_user(text: str) -> dict:
    response = llm.complete(f"Extract user info as JSON: {text}")
    return json.loads(response) # ❌ Fails 20-40% of the time

user = extract_user("Contact Alice at alice@example.com")
print(user['email']) # ❌ KeyError if LLM used 'e-mail' instead
```

Key insight: Without formal constraints, LLMs produce:

- Inconsistent field names (`email` vs `e-mail` vs `emailAddress`)
- Wrong types (`"age": "25"` string instead of integer)
- Missing required fields
- Extra unexpected fields
- Invalid JSON syntax (trailing commas, single quotes)

Production solution:

python

```
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
import instructor
from openai import OpenAI

# 1. Define formal schema
class User(BaseModel):
    name: str = Field(..., description="Full name of the user")
    email: EmailStr = Field(..., description="Valid email address")
    age: Optional[int] = Field(None, ge=0, le=150, description="Age in years")

    class Config:
        # Validate on assignment
        validate_assignment = True

# 2. Use library that enforces schema
client = instructor.from_openai(OpenAI())

def extract_user(text: str) -> User:
    """Type-safe extraction with runtime validation"""
    return client.chat.completions.create(
        model="gpt-4",
        response_model=User,
        messages=[{"role": "user", "content": f"Extract user info: {text}"}]
    )

# 3. Type-safe usage
user = extract_user("Alice, 25, contact at alice@example.com")
print(user.email) # ✅ Guaranteed to exist and be valid email
print(user.age + 1) # ✅ Guaranteed to be int (or None)
```

Why this matters: Downstream systems crash on malformed data. Schema enforcement = reliability.

Perspective 2: Product Engineer

Structured outputs enable composability. One LLM's output becomes another's input.

The Composability Problem:

python

```
# Step 1: Analyze customer feedback
analysis = llm_analyze(feedback)
# Output: "The customer is frustrated with billing. Sentiment: negative. Priority: high"

# Step 2: Create support ticket
ticket = llm_create_ticket(analysis) # ❌ Fails to parse unstructured text
```

Solution: Structured pipelines

python

```
from pydantic import BaseModel
from enum import Enum

class Sentiment(str, Enum):
    POSITIVE = "positive"
    NEGATIVE = "negative"
    NEUTRAL = "neutral"

class Priority(str, Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    URGENT = "urgent"

class FeedbackAnalysis(BaseModel):
    summary: str
    sentiment: Sentiment
    priority: Priority
    category: str
    key_issues: list[str]

class SupportTicket(BaseModel):
    title: str
    description: str
    priority: Priority
    tags: list[str]

# Step 1: Structured analysis
analysis: FeedbackAnalysis = llm_analyze(feedback)

# Step 2: Compose - use structured data directly
ticket: SupportTicket = llm_create_ticket(
    summary=analysis.summary,
```

```

    sentiment=analysis.sentiment,
    priority=analysis.priority,
    issues=analysis.key_issues
)

# ✅ Type-safe, composable, reliable

```

Why this matters: Multi-step LLM workflows require structured interfaces. Ad-hoc text passing breaks down at scale.

Perspective 3: Researcher

LLMs generate text token-by-token. JSON is a serialization format, not a generation format. This mismatch causes failures.

Advanced insight: Tokenization breaks JSON structure

python

```

import tiktoken

# What we want
desired_json = '{"name": "Alice", "age": 25}'

# How LLM generates it (token by token)
enc = tiktoken.encoding_for_model("gpt-4")
tokens = enc.encode(desired_json)
print([enc.decode([t]) for t in tokens])

# Output (token boundaries):
# ['{', '"', 'name', ':", ' ', '"', 'Al', 'ice', '"', ',', ' ', '"', 'age', ':", ' ', ' ', '25', '}', ']']
#
#           ^^^^^^^^^^^^^
#           Note: "Alice" splits across 2 tokens!

# Problem: At each token, LLM must "remember" it's inside JSON
# If it forgets → generates invalid JSON

# Common failures at token boundaries:
failures = [
    '{"name": "Alice", "age": 25,}', # Trailing comma (forgot to check if last)
    '{"name": "Alice" "age": 25}',  # Missing comma (forgot separator)
    '{"name': 'Alice'}',            # Single quotes (wrong context)
    '{"name": "Alice"',             # Unclosed (ran out of tokens)
]

```

Why constrained decoding helps:

python

```
# Traditional sampling: At each token, sample from all possible tokens
def generate_next_token(context):
    logits = model(context)
    next_token = sample(logits) # ❌ Could be any token (including invalid JSON)
    return next_token

# Constrained decoding: Only sample from JSON-valid tokens
def generate_next_token_constrained(context, json_state):
    logits = model(context)

    # Determine what tokens are valid given current JSON state
    if json_state == "in_string":
        valid_tokens = ALPHANUMERIC_TOKENS + ['"'] # Can't have { } here
    elif json_state == "after_key":
        valid_tokens = [':'] # Must be colon
    elif json_state == "after_value":
        valid_tokens = [',', '}'] # Must be comma or close brace

    # Mask out invalid tokens
    masked_logits = mask(logits, valid_tokens)
    next_token = sample(masked_logits) # ✅ Guaranteed valid

    return next_token
```

Tradeoff: Constrained decoding guarantees validity but:

- Slower (must track parse state)
- Requires custom inference code (not available on all APIs)
- May reduce output quality (restricts model's choices)

Perspective 4: Hiring Manager

Show me you've debugged structured output failures in production and built robust extraction.

Red flags:

- No validation layer (assumes LLM output is always valid)
- Single extraction attempt (no retry logic)
- Hard-coded field names (breaks on variation)
- No schema versioning (can't evolve over time)

- Regex parsing (brittle)

Green flags:

- Pydantic models with rich validation (email, dates, enums)
 - Multi-strategy fallbacks (function calling → JSON mode → parsing)
 - Partial extraction on failures (get what you can)
 - Schema documentation and versioning
 - Quantified success rates (98% valid outputs)
-

Advanced Production Use Cases

Case 1: Multi-Level Nested Schemas

Problem: Complex domain objects with deep nesting.

Solution: Compose Pydantic models hierarchically.

python

```
from pydantic import BaseModel, Field, validator
from typing import List, Optional
from datetime import datetime

class Address(BaseModel):
    street: str
    city: str
    state: str
    zip_code: str = Field(..., regex=r'^\d{5}(-\d{4})?$')
    country: str = "USA"

class PhoneNumber(BaseModel):
    number: str = Field(..., regex=r'^\+?1?\d{10,15}$')
    type: str = Field(..., regex=r'^(mobile|home|work)$')

class Person(BaseModel):
    name: str = Field(..., min_length=1, max_length=100)
    email: EmailStr
    age: Optional[int] = Field(None, ge=0, le=150)
    addresses: List[Address] = []
    phone_numbers: List[PhoneNumber] = []
```



```

@validator('phone_numbers')
def validate_phone_numbers(cls, v):
    if len(v) > 5:
        raise ValueError('Maximum 5 phone numbers allowed')
    return v

class Company(BaseModel):
    name: str
    founded: datetime
    employees: List[Person]
    headquarters: Address

    @validator('employees')
    def validate_employees(cls, v):
        if not v:
            raise ValueError('Company must have at least one employee')
        return v

# Usage with instructor
client = instructor.from_openai(OpenAI())

company: Company = client.chat.completions.create(
    model="gpt-4",
    response_model=Company,
    messages=[{
        "role": "user",
        "content": """
        Extract company info:

        Acme Corp was founded on Jan 1, 2020. Headquarters at
        123 Main St, San Francisco, CA 94105.

        Employees:
        - Alice (alice@acme.com, 30 years old, mobile: 555-1234)
        - Bob (bob@acme.com, mobile: 555-5678)
        """
    }]
)

# ✅ Fully validated nested structure
print(company.headquarters.city) # "San Francisco"
print(company.employees[0].phone_numbers[0].number) # "555-1234"
print(company.employees[0].phone_numbers[0].type) # "mobile"

```

Impact: Handles complex real-world data structures with type safety and validation at every level.

Case 2: Union Types for Polymorphic Responses

Problem: LLM should return different schema based on input.

Solution: Use discriminated unions.

python

```
from pydantic import BaseModel, Field
from typing import Union, Literal

class SuccessResponse(BaseModel):
    status: Literal["success"]
    data: dict
    message: str

class ErrorResponse(BaseModel):
    status: Literal["error"]
    error_code: str
    error_message: str
    retry_after: Optional[int] = None

class PendingResponse(BaseModel):
    status: Literal["pending"]
    job_id: str
    estimated_completion: datetime

# Discriminated union
Response = Union[SuccessResponse, ErrorResponse, PendingResponse]

# Instructor handles discriminated unions
result: Response = client.chat.completions.create(
    model="gpt-4",
    response_model=Response,
    messages=[{"role": "user", "content": "Process this request: ..."}]
)

# Type-safe pattern matching
if isinstance(result, SuccessResponse):
    print(result.data)
elif isinstance(result, ErrorResponse):
    logger.error(f"Error {result.error_code}: {result.error_message}")
    if result.retry_after:
```

```
        time.sleep(result.retry_after)
    elif isinstance(result, PendingResponse):
        job_status = poll_job(result.job_id)
```

Why this works: Pydantic uses `status` field to discriminate which schema to validate against.

Case 3: Partial Extraction with Fallbacks

Problem: When extraction fails, you lose all data.

Solution: Multi-tier extraction strategy.

python

```
class StrictProduct(BaseModel):
    """Ideal schema - all fields required"""
    name: str = Field(..., min_length=1)
    price: float = Field(..., gt=0)
    category: str
    description: str
    in_stock: bool
    sku: str = Field(..., regex=r'^[A-Z0-9]{8,12}$')

class RelaxedProduct(BaseModel):
    """Fallback schema - only critical fields required"""
    name: str
    price: Optional[float] = None
    category: Optional[str] = None
    description: Optional[str] = None
    in_stock: Optional[bool] = None
    sku: Optional[str] = None

class ExtractionResult(BaseModel):
    """Wrapper with metadata"""
    product: Union[StrictProduct, RelaxedProduct]
    extraction_quality: Literal["high", "medium", "low"]
    missing_fields: List[str]

def extract_product_robust(text: str) -> ExtractionResult:
    """Multi-tier extraction with fallbacks"""

    # Attempt 1: Strict extraction
    try:
        product = client.chat.completions.create(
            model="gpt-4",
```

```

        response_model=StrictProduct,
        messages=[{"role": "user", "content": f"Extract product: {text}"}]
    )
    return ExtractionResult(
        product=product,
        extraction_quality="high",
        missing_fields=[]
    )
except ValidationError as e:
    logger.warning(f"Strict extraction failed: {e}")

# Attempt 2: Relaxed extraction
try:
    product = client.chat.completions.create(
        model="gpt-4",
        response_model=RelaxedProduct,
        messages=[{
            "role": "user",
            "content": f"Extract product info (extract what you can): {text}"
        }]
    )

    # Identify missing fields
    missing = [
        field for field, value in product.dict().items()
        if value is None
    ]

    return ExtractionResult(
        product=product,
        extraction_quality="medium" if len(missing) < 3 else "low",
        missing_fields=missing
    )
except Exception as e:
    logger.error(f"All extraction attempts failed: {e}")

# Attempt 3: Manual parsing fallback
product = manual_fallback_parser(text)
return ExtractionResult(
    product=product,
    extraction_quality="low",
    missing_fields=list(product.dict().keys())
)

# Usage
result = extract_product_robust(messy_product_text)

```

```

if result.extraction_quality == "high":
    # All fields present and validated
    db.insert_product(result.product)
elif result.extraction_quality == "medium":
    # Some fields missing - flag for review
    db.insert_product_draft(result.product)
    flag_for_human_review(result.product, result.missing_fields)
else:
    # Low quality - requires manual entry
    queue_for_manual_entry(text)

```

Impact: 95%+ extraction success rate vs 65% with single-attempt approach.

Expert-Level Comprehension Tests

Question 1: Your LLM reliably generates valid JSON 95% of the time when you test it. In production, valid JSON rate drops to 60%. What are your top 3 hypotheses and diagnostic approaches?

Expected Answer

Hypotheses: 1. **Input distribution shift** (most common) - Test data: Clean, well-formatted examples - Production: Messy, noisy, edge cases **Diagnosis:** python

```

# Sample production failures
failures = get_failed_extractions(limit=100)

# Analyze characteristics
print(f"Avg length: {np.mean([len(f.input) for f in failures])}")
print(f"Special chars: {sum('0' in f.input for f in failures)}")
print(f"Non-ASCII: {sum(any(ord(c) > 127 for c in f.input) for f in failures)}")

# Common pattern in failures?

```

Fix: Add adversarial examples to training data, use constrained decoding 2. **Truncation** due to token limits - Test: Short inputs that fit in context - Production: Long inputs that get truncated **Diagnosis:** python

```

# Check if failures correlate with input length
failures_by_length = df.groupby(
    pd.cut(df['input_length'], bins=[0, 1000, 2000, 4000, 8000])

```

```

) ['failure_rate'].mean()

print(failures_by_length)
# If failure rate increases with length → truncation issue

```

****Fix:**** Implement sliding window extraction, chunk large inputs 3. ****Temperature > 0 causing stochastic failures**** - Test: Used temperature=0 (deterministic) - Production: Someone set temperature=0.7 (more creative but less reliable) ****Diagnosis:**** python

```

# Check LLM call logs
print(df.groupby('temperature')['failure_rate'].mean())
# Temperature 0.0: 5% failure
# Temperature 0.7: 40% failure ← smoking gun

```

****Fix:**** Force temperature=0 for structured outputs ****Anti-pattern:**** "The model got worse" (no systematic diagnosis)

Question 2: You need to extract 20 fields from documents. Using a single Pydantic model with 20 fields has 40% success rate. How would you redesign for >90% success?

Expected Answer

****Problem:**** Large schemas overwhelm the LLM. It forgets fields, gets confused, hallucinates.

****Solution 1: Hierarchical extraction**** python

```

# Instead of single 20-field model
class Document(BaseModel):
    field1: str
    field2: str
    # ... 18 more fields
    field20: str

# Use hierarchical groups
class PersonalInfo(BaseModel):
    name: str
    age: int
    email: EmailStr

class AddressInfo(BaseModel):
    street: str
    city: str
    state: str
    zip: str

class FinancialInfo(BaseModel):

```

```

    income: float
    tax_id: str
    bank_account: str

class Document(BaseModel):
    personal: PersonalInfo
    address: AddressInfo
    financial: FinancialInfo
    # ... other logical groups

# Extract in stages
personal = extract(text, PersonalInfo) # 3 fields - easy
address = extract(text, AddressInfo)   # 4 fields - easy
financial = extract(text, FinancialInfo) # 3 fields - easy

document = Document(
    personal=personal,
    address=address,
    financial=financial
)
# Success rate: 95%+

```

****Solution 2: Multi-pass extraction**** python

```

# Pass 1: Extract critical fields only
class CoreFields(BaseModel):
    name: str
    document_id: str
    date: datetime

core = extract(text, CoreFields)

# Pass 2: Extract details (with core as context)
class DetailedFields(BaseModel):
    # Remaining 17 fields
    pass

details = extract(
    text,
    DetailedFields,
    context=f"Document: {core.name}, ID: {core.document_id}"
)

```

****Solution 3: Iterative extraction with validation**** python

```

extracted_fields = {}

for field_name, field_type in schema.items():
    # Extract one field at a time
    value = extract_single_field(text, field_name, field_type)

    if validate(value):
        extracted_fields[field_name] = value
    else:
        # Retry with more context
        value = extract_single_field(
            text,
            field_name,
            field_type,
            context=f"Already extracted: {extracted_fields}"
        )
        extracted_fields[field_name] = value

document = Document(**extracted_fields)

```

****Measurement:**** python

```

# Test on 100 documents
results = []
for doc in test_docs:
    # Single-pass
    success_single = try_extract_all_at_once(doc)

    # Hierarchical
    success_hierarchical = try_extract_hierarchical(doc)

    results.append({
        'single_pass': success_single,
        'hierarchical': success_hierarchical
    })

df = pd.DataFrame(results)
print(df.mean())
# single_pass: 0.42
# hierarchical: 0.94

```

****Key insight:**** LLM attention is limited. Break complex tasks into simpler sub-tasks.

Question 3: Your schema has an enum field `status: Literal["pending", "approved", "rejected"]`. The LLM generates "Pending" (capitalized) 30% of the time, causing validation errors. What are 3 different ways to fix this?

Expected Answer

****Fix 1: Pydantic validator to normalize**** python

```
from pydantic import BaseModel, validator

class Application(BaseModel):
    status: Literal["pending", "approved", "rejected"]

    @validator('status', pre=True)
    def normalize_status(cls, v):
        if isinstance(v, str):
            return v.lower()
        return v

# Now accepts "Pending", "PENDING", "pending" → all become "pending"
```

****Fix 2: Make prompt more explicit**** python

```
system_prompt = """
Extract application status. IMPORTANT:
- Use EXACTLY these values (lowercase): "pending", "approved", "rejected"
- Do NOT capitalize
- Examples:
  ✓ "pending"
  x "Pending"
  x "PENDING"
"""
```

****Fix 3: Use constrained decoding to force lowercase**** python

```
from outlines import models, generate

model = models.transformers("mistralai/Mistral-7B-v0.1")

# Define exact allowed values
status_values = ["pending", "approved", "rejected"]

# Constrained generation - can ONLY produce these exact strings
generator = generate.choice(model, status_values)
```

```
status = generator("What is the application status?")
# Guaranteed to be one of the three exact values
```

****Fix 4: Use function calling (most reliable)**** python

```
# OpenAI function calling enforces enum values
function = {
    "name": "extract_application",
    "parameters": {
        "type": "object",
        "properties": {
            "status": {
                "type": "string",
                "enum": ["pending", "approved", "rejected"]
                # OpenAI enforces this at generation time
            }
        }
    }
}

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": "Extract status from: ..."}],
    functions=[function],
    function_call={"name": "extract_application"}
)

# Guaranteed to be one of the enum values
```

****Comparison:**** | Method | Reliability | Latency | Compatibility | | ----- |
----- | ----- | ----- | | Validator | 100% (post-generation) | Fast | All
models | | Explicit prompt | 70-90% | Fast | All models | | Constrained decoding | 100% (during
generation) | Slow (2-3x) | Local models only | | Function calling | 99%+ | Fast | OpenAI,
Anthropic | ****Recommendation:**** Function calling for production (best balance of reliability and
speed).

3. Fastest Path to Competence (Execution-Focused)

Prerequisites (Week 0: Setup)

Install:

bash

```
pip install \
  pydantic \           # Schema validation
  instructor \         # OpenAI function calling wrapper
  openai anthropic \   # LLM APIs
  outlines \           # Constrained generation
  jsonschema \         # Schema validation
  pytest \             # Testing
  pandas \             # Analysis
  email-validator       # Email validation for Pydantic
```

Repository structure:

```
structured-outputs/
├── schemas/
│   ├── base.py          # Core Pydantic models
│   ├── complex.py       # Nested schemas
│   └── versioned/       # Schema versions
├── extractors/
│   ├── function_calling.py # OpenAI function calling
│   ├── json_mode.py      # JSON mode
│   ├── constrained.py    # Grammar-constrained
│   └── fallback.py       # Multi-strategy
├── tests/
│   ├── test_schemas.py
│   └── test_extraction.py
├── data/
│   ├── test_cases.json
│   └── extraction_results.csv
└── README.md
```

Week 1: Pydantic Mastery & Schema Design

Learning Objectives:

- Design robust Pydantic schemas
- Implement validation at multiple levels
- Handle nested and complex types
- Version schemas for evolution

Day 1-2: Core Pydantic Patterns

Hands-on Lab:

python

```
# schemas/base.py
from pydantic import BaseModel, Field, EmailStr, validator, root_validator
from typing import Optional, List, Literal
from datetime import datetime
from enum import Enum

class Priority(str, Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    URGENT = "urgent"

class Task(BaseModel):
    """Basic task schema with validation"""

    title: str = Field(
        ..., # Required
        min_length=1,
        max_length=200,
        description="Task title"
    )

    description: Optional[str] = Field(
        None,
        max_length=2000,
        description="Detailed description"
    )

    priority: Priority = Field(
        default=Priority.MEDIUM,
```

```

        description="Task priority level"
    )

    due_date: Optional[datetime] = Field(
        None,
        description="Due date (ISO format)"
    )

    tags: List[str] = Field(
        default_factory=list,
        description="List of tags"
    )

    assignee_email: Optional[EmailStr] = Field(
        None,
        description="Assignee's email"
    )

    estimated_hours: Optional[float] = Field(
        None,
        ge=0,
        le=1000,
        description="Estimated hours to complete"
    )

    # Field-level validation
    @validator('tags')
    def validate_tags(cls, v):
        if len(v) > 10:
            raise ValueError('Maximum 10 tags allowed')
        # Normalize tags
        return [tag.lower().strip() for tag in v]

    @validator('due_date')
    def validate_due_date(cls, v):
        if v and v < datetime.now():
            raise ValueError('Due date cannot be in the past')
        return v

    # Cross-field validation
    @root_validator
    def validate_urgent_tasks(cls, values):
        priority = values.get('priority')
        due_date = values.get('due_date')

        if priority == Priority.URGENT and not due_date:

```

```

        raise ValueError('Urgent tasks must have a due date')

    return values

class Config:
    # Customize behavior
    use_enum_values = True # Serialize enums as values
    validate_assignment = True # Validate on attribute assignment
    json_encoders = {
        datetime: lambda v: v.isoformat()
    }

# Usage
task = Task(
    title="Fix authentication bug",
    priority=Priority.HIGH,
    tags=["bug", "security", "auth"],
    due_date=datetime(2025, 3, 1),
    estimated_hours=8
)

print(task.json(indent=2))
# Valid JSON with all validations passed

# Validation in action
try:
    invalid_task = Task(
        title="", # Too short
        priority=Priority.URGENT,
        # Missing due_date for urgent task
    )
except ValidationError as e:
    print(e.json())
    # Shows all validation errors

```

Deliverable: 5+ Pydantic models with:

- Field-level constraints
- Custom validators
- Cross-field validation
- Rich type annotations

Resources:

- [Pydantic Documentation](#)

- [Pydantic V2 Migration Guide](#)

Day 3-4: Nested Schemas & Complex Types

Hands-on Lab:

python

```
# schemas/complex.py
from pydantic import BaseModel, Field, validator
from typing import List, Union, Optional, Dict, Any
from datetime import datetime

class Author(BaseModel):
    name: str
    email: EmailStr
    affiliation: Optional[str] = None

class Citation(BaseModel):
    title: str
    authors: List[str]
    year: int = Field(..., ge=1900, le=2030)
    doi: Optional[str] = None

class Section(BaseModel):
    heading: str
    content: str
    subsections: List['Section'] = [] # Recursive!

    @validator('subsections')
    def validate_depth(cls, v, values):
        # Prevent excessive nesting
        def max_depth(sections, current=0):
            if not sections:
                return current
            return max(max_depth(s.subsections, current + 1) for s in sections)

        if max_depth(v) > 5:
            raise ValueError('Maximum nesting depth is 5')
        return v

    # Update forward references for recursive models
    Section.update_forward_refs()

class ResearchPaper(BaseModel):
```

```

title: str = Field(..., min_length=10, max_length=300)
authors: List[Author] = Field(..., min_items=1)
abstract: str = Field(..., min_length=100, max_length=2000)
sections: List[Section]
citations: List[Citation] = []
keywords: List[str] = Field(..., min_items=1, max_items=10)
published_date: datetime

@validator('sections')
def validate_sections(cls, v):
    required_sections = {'introduction', 'methodology', 'results', 'conclusion'}
    section_headings = {s.heading.lower() for s in v}

    missing = required_sections - section_headings
    if missing:
        raise ValueError(f'Missing required sections: {missing}')

    return v

def count_words(self) -> int:
    """Utility method"""
    total = len(self.abstract.split())
    for section in self.sections:
        total += len(section.content.split())
    return total

# Usage with instructor
import instructor
from openai import OpenAI

client = instructor.from_openai(OpenAI())

paper: ResearchPaper = client.chat.completions.create(
    model="gpt-4",
    response_model=ResearchPaper,
    messages=[{
        "role": "user",
        "content": """
        Extract structured information from this research paper:

        [Paper text here...]
        """
    }],
    max_retries=3 # Retry on validation errors
)

```



```
print(f"Paper: {paper.title}")
print(f"Authors: {' '.join(a.name for a in paper.authors)}")
print(f"Word count: {paper.count_words()}")
print(f"Sections: {len(paper.sections)}")
```

Deliverable: Complex nested schema (3+ levels deep) with:

- Recursive types
- Cross-field validation
- Utility methods

Day 5-7: Schema Versioning & Evolution

Hands-on Lab:

python

```
# schemas/versioned/task_v1.py
class TaskV1(BaseModel):
    title: str
    description: str
    priority: str

# schemas/versioned/task_v2.py
class TaskV2(BaseModel):
    title: str
    description: Optional[str] = None # Made optional
    priority: Priority # Changed to enum
    due_date: Optional[datetime] = None # New field

    @classmethod
    def from_v1(cls, task_v1: TaskV1) -> 'TaskV2':
        """Migrate from V1 to V2"""
        # Map string priority to enum
        priority_map = {
            'low': Priority.LOW,
            'medium': Priority.MEDIUM,
            'high': Priority.HIGH
        }

        return cls(
            title=task_v1.title,
            description=task_v1.description,
            priority=priority_map.get(task_v1.priority.lower(), Priority.MEDIUM)
        )
```

```

# schemas/versioned/task_v3.py
class TaskV3(BaseModel):
    title: str
    description: Optional[str] = None
    priority: Priority
    due_date: Optional[datetime] = None
    assignee: Optional[str] = None # New field
    tags: List[str] = [] # New field

    @classmethod
    def from_v2(cls, task_v2: TaskV2) -> 'TaskV3':
        """Migrate from V2 to V3"""
        return cls(
            title=task_v2.title,
            description=task_v2.description,
            priority=task_v2.priority,
            due_date=task_v2.due_date,
            assignee=None, # New field, default None
            tags=[]
        )

# Version manager
class SchemaVersionManager:
    VERSIONS = {
        'v1': TaskV1,
        'v2': TaskV2,
        'v3': TaskV3
    }

    MIGRATIONS = {
        ('v1', 'v2'): TaskV2.from_v1,
        ('v2', 'v3'): TaskV3.from_v2
    }

    CURRENT_VERSION = 'v3'

    @classmethod
    def migrate(cls, data: dict, from_version: str, to_version: str = None):
        """Migrate data from one version to another"""
        if to_version is None:
            to_version = cls.CURRENT_VERSION

        if from_version == to_version:
            return cls.VERSIONS[to_version](**data)

```

```

    # Parse as source version
    source_schema = cls.VERSIONS[from_version]
    source_obj = source_schema(**data)

    # Apply migrations
    current_version = from_version
    current_obj = source_obj

    while current_version != to_version:
        # Find next migration
        next_version = cls._get_next_version(current_version, to_version)
        migration_fn = cls.MIGRATIONS[(current_version, next_version)]

        current_obj = migration_fn(current_obj)
        current_version = next_version

    return current_obj

    @classmethod
    def _get_next_version(cls, current: str, target: str) -> str:
        """Get next version in migration path"""
        versions = list(cls.VERSIONS.keys())
        current_idx = versions.index(current)
        target_idx = versions.index(target)

        if current_idx < target_idx:
            return versions[current_idx + 1]
        else:
            raise ValueError(f"Cannot migrate backwards from {current} to {target}")

# Usage
old_task_data = {
    'title': 'Fix bug',
    'description': 'Auth issue',
    'priority': 'high'
}

# Migrate from v1 to v3
task_v3 = SchemaVersionManager.migrate(old_task_data, from_version='v1')
print(task_v3)
# TaskV3(title='Fix bug', description='Auth issue', priority=<Priority.HIGH: 'high'>, ...)

```

Deliverable: Versioning system with:

- 3+ schema versions

- Migration functions
 - Version manager
 - Backward compatibility tests
-

Week 2: Function Calling & JSON Mode

Learning Objectives:

- Implement OpenAI function calling
- Use Anthropic tool use
- Handle parallel function calls
- Build error recovery

Day 8-10: OpenAI Function Calling

Hands-on Lab:

python

```
# extractors/function_calling.py
import instructor
from openai import OpenAI
from pydantic import BaseModel, Field
from typing import List, Optional

client = instructor.from_openai(OpenAI())

class MovieReview(BaseModel):
    """Structured movie review"""
    title: str = Field(description="Movie title")
    rating: float = Field(ge=0, le=10, description="Rating out of 10")
    genre: List[str] = Field(description="List of genres")
    pros: List[str] = Field(description="Positive aspects")
    cons: List[str] = Field(description="Negative aspects")
    recommendation: bool = Field(description="Would you recommend this movie?")

def extract_movie_review(text: str) -> MovieReview:
    """Extract structured review using function calling"""
    return client.chat.completions.create(
        model="gpt-4",
        response_model=MovieReview,
        messages=[
```

```

        {
            "role": "system",
            "content": "Extract structured movie review from user text."
        },
        {
            "role": "user",
            "content": text
        }
    ],
    max_retries=3 # Automatically retry on validation errors
)

# Usage
review_text = """
I just watched Inception. Amazing movie! The plot was mind-bending
and the visuals were stunning. Rating it 9/10. My only complaint
is it was a bit confusing at times. Definitely recommend it to
sci-fi fans.
"""

review = extract_movie_review(review_text)
print(review.model_dump_json(indent=2))

# Output:
# {
#   "title": "Inception",
#   "rating": 9.0,
#   "genre": ["sci-fi", "thriller"],
#   "pros": ["mind-bending plot", "stunning visuals"],
#   "cons": ["confusing at times"],
#   "recommendation": true
# }

```

Advanced: Parallel function calls

python

```

from typing import List

class SearchQuery(BaseModel):
    query: str
    filters: Optional[Dict[str, Any]] = None

class MultipleSearchQueries(BaseModel):
    """Multiple search queries to execute in parallel"""

```

```

queries: List[SearchQuery] = Field(
    ...,
    description="List of search queries to execute"
)

def generate_search_queries(user_request: str) -> MultipleSearchQueries:
    """Generate multiple search queries for comprehensive results"""
    return client.chat.completions.create(
        model="gpt-4",
        response_model=MultipleSearchQueries,
        messages=[{
            "role": "user",
            "content": f"""
            Generate 3-5 diverse search queries to answer this request:
            {user_request}

            Include different angles and phrasings.
            """
        }]
    )

# Usage
queries = generate_search_queries(
    "What are the best practices for LLM prompt engineering?"
)

# Execute searches in parallel
import asyncio

async def search_all(queries: List[SearchQuery]):
    tasks = [search_async(q.query, q.filters) for q in queries]
    return await asyncio.gather(*tasks)

results = asyncio.run(search_all(queries.queries))

```

Deliverable: Function calling implementation with:

- Single function extraction
- Parallel function calls
- Automatic retry logic
- Error handling

Day 11-12: JSON Mode & Constrained Generation

Hands-on Lab:

python

```
# extractors/json_mode.py
import json
from openai import OpenAI

client = OpenAI()

def extract_with_json_mode(text: str, schema: dict) -> dict:
    """Use OpenAI JSON mode for guaranteed valid JSON"""

    response = client.chat.completions.create(
        model="gpt-4-turbo",
        response_format={"type": "json_object"}, # Enables JSON mode
        messages=[
            {
                "role": "system",
                "content": f"""
                Extract information as JSON matching this schema:
                {json.dumps(schema, indent=2)}

                Return ONLY valid JSON, no other text.
                """
            },
            {
                "role": "user",
                "content": text
            }
        ]
    )

    # Parse JSON (guaranteed valid by JSON mode)
    result = json.loads(response.choices[0].message.content)

    # Validate against schema
    from jsonschema import validate
    validate(instance=result, schema=schema)

    return result

# Usage
schema = {
```

```

    "type": "object",
    "properties": {
        "name": {"type": "string"},
        "age": {"type": "integer", "minimum": 0, "maximum": 150},
        "email": {"type": "string", "format": "email"}
    },
    "required": ["name", "email"]
}

result = extract_with_json_mode("Alice, 30, alice@example.com", schema)
print(result)
# {'name': 'Alice', 'age': 30, 'email': 'alice@example.com'}

```

Constrained generation with Outlines:

python

```

# extractors/constrained.py
from outlines import models, generate
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int
    city: str

# Load model (local inference required)
model = models.transformers("mistralai/Mistral-7B-v0.1")

# Create constrained generator from Pydantic schema
generator = generate.json(model, Person)

# Generate - output GUARANTEED to match schema
prompt = "Extract person info: Alice, 30, lives in San Francisco"
person_json = generator(prompt)

# Parse into Pydantic model
person = Person.parse_raw(person_json)
print(person)

```

Comparison table:

python


```

# extractors/comparison.py
import time
from typing import Callable

def benchmark_extraction_methods(test_cases: List[str]):
    """Compare different extraction methods"""

    methods = {
        'function_calling': extract_with_function_calling,
        'json_mode': extract_with_json_mode,
        'constrained': extract_with_constrained,
        'prompt_only': extract_with_prompt_only
    }

    results = []

    for method_name, method_fn in methods.items():
        for text in test_cases:
            start = time.time()

            try:
                result = method_fn(text)
                success = True
                error = None
            except Exception as e:
                success = False
                error = str(e)

            latency = time.time() - start

            results.append({
                'method': method_name,
                'success': success,
                'latency': latency,
                'error': error
            })

    df = pd.DataFrame(results)

    summary = df.groupby('method').agg({
        'success': 'mean',
        'latency': 'mean'
    }).round(3)

    print(summary)

```

```

    return summary

# Run benchmark
test_cases = load_test_cases() # 100 diverse examples
benchmark_extraction_methods(test_cases)

# Expected output:
#
#           success  latency
# function_calling    0.98    0.850
# json_mode          0.95    0.720
# constrained         1.00    2.300 # Slower but 100% reliable
# prompt_only        0.65    0.650 # Fast but unreliable

```

Deliverable: Comparison showing:

- Success rate per method
- Latency per method
- Recommendation matrix (when to use each)

Day 13-14: Error Recovery & Fallbacks

Hands-on Lab:

python

```

# extractors/fallback.py
from typing import Optional, Union
from pydantic import BaseModel, ValidationError
import logging

logger = logging.getLogger(__name__)

class ExtractionStrategy(str, Enum):
    FUNCTION_CALLING = "function_calling"
    JSON_MODE = "json_mode"
    CONSTRAINED = "constrained"
    PARSING = "parsing"

class ExtractionResult(BaseModel):
    data: Optional[BaseModel]
    strategy_used: ExtractionStrategy
    success: bool
    attempts: int
    errors: List[str]

```

```

class RobustExtractor:
    """Multi-strategy extractor with fallbacks"""

    def __init__(self, schema: type[BaseModel]):
        self.schema = schema
        self.strategies = [
            (ExtractionStrategy.FUNCTION_CALLING, self._extract_function_calling),
            (ExtractionStrategy.JSON_MODE, self._extract_json_mode),
            (ExtractionStrategy.PARSING, self._extract_with_parsing)
        ]

    def extract(self, text: str, max_attempts: int = 3) -> ExtractionResult:
        """Try multiple strategies until one succeeds"""
        errors = []
        attempts = 0

        for strategy_name, strategy_fn in self.strategies:
            for attempt in range(max_attempts):
                attempts += 1

                try:
                    logger.info(f"Attempting {strategy_name}, attempt {attempt + 1}")
                    data = strategy_fn(text)

                    # Validate
                    validated = self.schema(**data.dict())

                    return ExtractionResult(
                        data=validated,
                        strategy_used=strategy_name,
                        success=True,
                        attempts=attempts,
                        errors=errors
                    )

                except ValidationError as e:
                    error_msg = f"{strategy_name} validation error: {e}"
                    logger.warning(error_msg)
                    errors.append(error_msg)

                    # Try with stricter prompt
                    if attempt < max_attempts - 1:
                        text = self._add_strict_instructions(text, e)

                except Exception as e:
                    error_msg = f"{strategy_name} failed: {e}"

```

```

        logger.error(error_msg)
        errors.append(error_msg)
        break # Move to next strategy

# All strategies failed
return ExtractionResult(
    data=None,
    strategy_used=ExtractionStrategy.PARSING,
    success=False,
    attempts=attempts,
    errors=errors
)

def _add_strict_instructions(self, text: str, error: ValidationError) -> str:
    """Add explicit instructions based on validation error"""
    error_fields = [e['loc'][0] for e in error.errors()]

    strict_instructions = f"""
    IMPORTANT: Previous attempt failed. Please ensure:
    - Field names EXACTLY match: {list(self.schema.__fields__.keys())}
    - Problem fields from last attempt: {error_fields}
    - Use correct types (string, number, boolean, array)
    - Required fields must not be null

    Original text: {text}
    """
    return strict_instructions

def _extract_function_calling(self, text: str):
    # Implementation...
    pass

def _extract_json_mode(self, text: str):
    # Implementation...
    pass

def _extract_with_parsing(self, text: str):
    """Fallback: try to parse unstructured output"""
    # Last resort: extract what we can
    pass

# Usage
extractor = RobustExtractor(schema=MovieReview)

result = extractor.extract("""
Great movie! The main character was so cool.

```

```
I'd give it 8 out of 10.
""")

if result.success:
    print(f"Extracted with {result.strategy_used} in {result.attempts} attempts")
    print(result.data)
else:
    print(f"Extraction failed after {result.attempts} attempts")
    print(f"Errors: {result.errors}")
```

Deliverable: Robust extraction with:

- 3+ fallback strategies
- Retry with improved prompts
- Detailed error logging
- Success rate >95% on diverse inputs

Week 3: Production Patterns & Portfolio

Learning Objectives:

- Handle streaming with structured outputs
- Implement partial extraction
- Build production-ready extraction service
- Create portfolio demonstrating mastery

Day 15-17: Streaming Structured Outputs

Hands-on Lab:

python

```
# extractors/streaming.py
import instructor
from openai import OpenAI
from pydantic import BaseModel
from typing import Iterable

client = instructor.from_openai(OpenAI(), mode=instructor.Mode.TOOLS_STRICT)

class ProgressUpdate(BaseModel):
```

```

    """Incremental extraction progress"""
    field_name: str
    field_value: Any
    confidence: float
    complete: bool

def extract_streaming(text: str, schema: type[BaseModel]) -> Iterable[ProgressUpdate]:
    """Stream extraction results as they become available"""

    # Use partial mode - get incomplete objects during generation
    extraction_stream = client.chat.completions.create_partial(
        model="gpt-4",
        response_model=schema,
        messages=[{"role": "user", "content": f"Extract: {text}"},],
        stream=True
    )

    for partial_obj in extraction_stream:
        # Get fields that have been populated
        for field_name, field_value in partial_obj.dict(exclude_none=True).items():
            yield ProgressUpdate(
                field_name=field_name,
                field_value=field_value,
                confidence=1.0 if field_value else 0.0,
                complete=partial_obj.dict() == schema(**partial_obj.dict()).dict()
            )

# Usage
class LongDocument(BaseModel):
    title: str
    author: str
    abstract: str
    sections: List[str]
    keywords: List[str]

long_text = "..." # 10,000 words

print("Streaming extraction...")
for update in extract_streaming(long_text, LongDocument):
    print(f"[{update.field_name}] {'✓' if update.complete else '...'}")

# Output shows progressive extraction:
# [title] ✓
# [author] ✓
# [abstract] ...
# [abstract] ✓

```

```
# [sections] ...  
# [sections] ✓  
# ...
```

Partial extraction on errors:

python

```
class PartialExtractor:  
    """Extract what's possible even if full extraction fails"""  
  
    def extract_best_effort(self, text: str, schema: type[BaseModel]) -> tuple[dict, list]:  
        """Return partial data + list of missing fields"""  
  
        try:  
            # Try full extraction  
            full_result = extract(text, schema)  
            return full_result.dict(), []  
  
        except ValidationError as e:  
            # Extract valid fields only  
            partial_data = {}  
            missing_fields = []  
  
            for field_name, field_type in schema.__fields__.items():  
                try:  
                    # Try to extract individual field  
                    value = self._extract_single_field(text, field_name, field_type)  
  
                    # Validate single field  
                    validated = field_type.type_(value)  
                    partial_data[field_name] = validated  
  
                except Exception:  
                    missing_fields.append(field_name)  
  
            return partial_data, missing_fields  
  
    def _extract_single_field(self, text: str, field_name: str, field_type) -> Any:  
        """Extract a single field"""  
        prompt = f"""  
From this text, extract only the {field_name}.  
Text: {text}  
  
Return ONLY the {field_name} value, nothing else.
```

```

"""

response = client.chat.completions.create(
    model="gpt-3.5-turbo", # Use cheap model for single fields
    messages=[{"role": "user", "content": prompt}]
)

return response.choices[0].message.content

# Usage
extractor = PartialExtractor()
partial_data, missing = extractor.extract_best_effort(messy_text, ComplexSchema)

print(f"Extracted {len(partial_data)}/{len(ComplexSchema.__fields__)} fields")
print(f"Missing: {missing}")

```

Deliverable: Streaming system with:

- Progressive field extraction
- Partial results on failure
- User feedback during extraction

Day 18-19: Production Service

Hands-on Lab:

Build complete extraction API:

python

```

# api/extraction_service.py
from fastapi import FastAPI, HTTPException, BackgroundTasks
from pydantic import BaseModel
from typing import Optional, Dict, Any
import uuid
import asyncio

app = FastAPI()

class ExtractionRequest(BaseModel):
    text: str
    schema_name: str
    options: Optional[Dict[str, Any]] = {}

class ExtractionResponse(BaseModel):

```



```

    job_id: str
    status: str
    result: Optional[Dict] = None
    error: Optional[str] = None

# Job store (use Redis in production)
jobs = {}

# Schema registry
SCHEMAS = {
    'person': Person,
    'task': Task,
    'review': MovieReview,
    # ... register all schemas
}

@app.post("/extract")
async def extract_async(
    request: ExtractionRequest,
    background_tasks: BackgroundTasks
):
    """Async extraction endpoint"""

    # Validate schema name
    if request.schema_name not in SCHEMAS:
        raise HTTPException(400, f"Unknown schema: {request.schema_name}")

    # Create job
    job_id = str(uuid.uuid4())
    jobs[job_id] = {
        'status': 'pending',
        'result': None,
        'error': None
    }

    # Start background extraction
    schema = SCHEMAS[request.schema_name]
    background_tasks.add_task(
        run_extraction,
        job_id,
        request.text,
        schema,
        request.options
    )

    return ExtractionResponse(

```

```

        job_id=job_id,
        status='pending'
    )

@app.get("/extract/{job_id}")
async def get_extraction_result(job_id: str):
    """Poll for extraction result"""

    if job_id not in jobs:
        raise HTTPException(404, "Job not found")

    job = jobs[job_id]

    return ExtractionResponse(
        job_id=job_id,
        status=job['status'],
        result=job['result'],
        error=job['error']
    )

async def run_extraction(
    job_id: str,
    text: str,
    schema: type[BaseModel],
    options: dict
):
    """Background extraction task"""

    try:
        jobs[job_id]['status'] = 'running'

        # Use robust extractor
        extractor = RobustExtractor(schema)
        result = extractor.extract(text, **options)

        if result.success:
            jobs[job_id] = {
                'status': 'completed',
                'result': result.data.dict(),
                'error': None
            }
        else:
            jobs[job_id] = {
                'status': 'failed',
                'result': None,
                'error': '; '.join(result.errors)
            }
    
```

```

    }

    except Exception as e:
        jobs[job_id] = {
            'status': 'failed',
            'result': None,
            'error': str(e)
        }

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

# Run: uvicorn api.extraction_service:app --reload

```

Client usage:

python

```

import requests
import time

# Submit extraction job
response = requests.post("http://localhost:8000/extract", json={
    "text": "Extract this: Alice, 30, alice@example.com",
    "schema_name": "person",
    "options": {"max_attempts": 3}
})

job_id = response.json()['job_id']
print(f"Job submitted: {job_id}")

# Poll for result
while True:
    response = requests.get(f"http://localhost:8000/extract/{job_id}")
    job = response.json()

    if job['status'] == 'completed':
        print("Extraction completed!")
        print(job['result'])
        break
    elif job['status'] == 'failed':
        print(f"Extraction failed: {job['error']}")
        break

```

```
time.sleep(1)
```

Deliverable: Production API with:

- Async extraction
- Job queuing
- Schema registry
- Error handling
- Health checks

Day 20-21: Portfolio Documentation

Create comprehensive README:

markdown

```
# Production Structured Output System

## Overview
Type-safe LLM extraction system achieving 98% success rate on complex schemas

## Architecture
```

Input Text → Strategy Selection → Extraction → Validation → Retry → Output

↓↓↓↓

Function Calling Pydantic Error Fallback

JSON Mode Validation Logging Strategies

Constrained Gen

```
## Results

### Extraction Success Rates
| Method | Simple Schemas | Complex Schemas | Nested Schemas |
|-----|-----|-----|-----|
| Prompt only | 65% | 35% | 15% |
| JSON mode | 92% | 78% | 60% |
| Function calling | 97% | 94% | 89% |
| **Multi-strategy (ours)** | **99%** | **98%** | **96%** |

### Performance Metrics
- Average latency: 1.2s (simple), 2.5s (complex)
- Partial extraction rate: 95% (when full extraction fails)
```

```
- API uptime: 99.8%

## Features

### 1. Schema-First Design
```python
class User(BaseModel):
 name: str
 email: EmailStr
 age: Optional[int] = Field(None, ge=0, le=150)

 @validator('name')
 def validate_name(cls, v):
 if len(v.split()) < 2:
 raise ValueError('Name must include first and last')
 return v
```

## 2. Multi-Strategy Extraction

Automatic fallback chain:

1. Function calling (98% success, fast)
2. JSON mode (95% success, fast)
3. Constrained generation (100% success, slow)
4. Partial extraction (graceful degradation)

## 3. Production API

```
Start service
uvicorn api.extraction_service:app

Extract
curl -X POST http://localhost:8000/extract \
 -H "Content-Type: application/json" \
 -d '{"text": "...", "schema_name": "person"}'
```

## Quality Assurance

---

Tested on 1000 diverse documents:

- News articles

- Research papers
- Customer reviews
- Support tickets
- Resumes

Metric	Value
Success rate	98.2%
Partial extraction rate	96.8%
Avg fields extracted	94.3%
False positives	0.3%

## Schema Examples

---

### Simple

```
class Task(BaseModel):
 title: str
 priority: Literal["low", "medium", "high"]
 due_date: Optional[datetime]
```

### Complex

```
class ResearchPaper(BaseModel):
 title: str
 authors: List[Author]
 sections: List[Section] # Recursive
 citations: List[Citation]
```

### With Validation

```
class Email(BaseModel):
 subject: str = Field(..., max_length=200)
```

```
body: str
recipients: List[EmailStr]

@validator('recipients')
def validate_recipients(cls, v):
 if len(v) > 100:
 raise ValueError('Max 100 recipients')
 return v
```

## Design Decisions

---

### Why Function Calling Over JSON Mode?

Function calling has higher success rate (97% vs 92%) and better error messages.

**Tradeoff:** Requires newer models (GPT-4, GPT-3.5-turbo-0613+)

### Why Multi-Strategy Fallbacks?

No single method is 100% reliable. Fallbacks ensure robustness.

**Tradeoff:** Added complexity, but worth it for production reliability.

### Why Pydantic Over JSON Schema?

Pydantic provides Python-native validation, IDE support, and automatic docs.

**Tradeoff:** Tied to Python ecosystem.

## Monitoring

---

- Real-time success rate dashboard
- Schema-level performance tracking
- Error pattern analysis
- Field-level extraction rates

## Future Work

---

1. Fine-tune routing model for strategy selection

2. Add semantic validation (not just type validation)
3. Support for more complex grammars (SQL, code)

...

...

---

## 4. Blind Spots & Underrated Perspectives

---

### What's Poorly Taught (Industry Reality Gaps)

#### Blind Spot 1: "Just Ask for JSON" Fails 20-40% of the Time

**Common teaching:** "Add 'respond in JSON format' to your prompt."

**Reality:** LLMs are text generators, not JSON generators. Without formal constraints, they make errors.

**Common JSON failures:**

python

```
Trailing commas
'{"name": "Alice", "age": 30,}' # Valid in Python, invalid JSON

Single quotes
'{'name': 'Alice'}' # Python dict syntax, invalid JSON

Missing quotes on keys
'{name: "Alice"}' # JavaScript-like, invalid JSON

Unescaped strings
'{"text": "She said "hello""}' # Broken by nested quotes

Incomplete generation
'{"name": "Alice", "age"' # Truncated by token limit

Extra text
'Here is the JSON: {"name": "Alice"}' # Preamble breaks parsing
```



```
Numbers as strings
'{"age": "30"}' # Type mismatch
```

### Measurement:

python

```
Test: "Just prompting" approach
prompt = "Extract as JSON: {text}"

success_count = 0
for text in test_cases: # 100 diverse examples
 response = llm.complete(prompt.format(text=text))
 try:
 json.loads(response)
 success_count += 1
 except json.JSONDecodeError:
 pass

print(f"Success rate: {success_count}%")
Typical result: 60-75% success rate
```

### Why this matters for hiring:

- "Just prompting" signals amateur
- Production systems need >98% reliability
- Strong engineers use formal constraints

## Blind Spot 2: Schema Design Greatly Affects Success Rates

**Common teaching:** "Define your schema and you're done."

**Reality:** Some schemas are much easier for LLMs to populate than others.

### Bad schema design:

python

```
class BadTask(BaseModel):
 task_title_string: str
 priority_level_enum: int # 0, 1, 2, 3
 is_complete_boolean: bool
 assigned_to_user_id: Optional[int]
 created_at_timestamp_unix: float
```

### Problems:

- Verbose field names confuse LLM
- Integer enum (no semantic meaning)
- Technical names ( `unix_timestamp` ) not in natural language

**Success rate: ~75%**

### Good schema design:

python

```
class GoodTask(BaseModel):
 title: str = Field(description="Brief task description")
 priority: Literal["low", "medium", "high", "urgent"]
 complete: bool = Field(description="Is the task finished?")
 assignee: Optional[str] = Field(None, description="Person responsible")
 created: datetime = Field(description="When task was created")
```

### Improvements:

- Concise field names
- String enum (semantic meaning)
- Natural language descriptions
- Standard types (datetime vs float)

**Success rate: ~95%**

### How strong engineers design schemas:

python

```
Principle 1: Use natural language field names
✓ "title" not "task_title_string"
✓ "complete" not "is_complete_boolean"

Principle 2: Use enums with semantic values
✓ Literal["low", "medium", "high"]
x int # 0, 1, 2, 3

Principle 3: Add descriptions as prompts
class Task(BaseModel):
 priority: str = Field(
 description="Priority level: low, medium, high, or urgent"
 # LLM reads this! Acts as implicit instruction
```

```

)

Principle 4: Keep nesting shallow
x 5 levels of nesting (hard for LLM to track)
✓ 2-3 levels max

Principle 5: Use standard types
✓ datetime (LLM knows ISO format)
x float (for timestamps - ambiguous)

```

### Blind Spot 3: Validation Errors Are Gold (Use Them!)

**Common teaching:** "Catch ValidationError and retry."

**Reality:** Validation errors contain precise information about what went wrong. Use them to improve the next attempt.

**Naive retry:**

python

```

for attempt in range(3):
 try:
 return extract(text)
 except ValidationError:
 continue # ❌ No learning between attempts

```

**Smart retry with error feedback:**

python

```

def extract_with_error_feedback(text: str, schema: type[BaseModel]) -> BaseModel:
 last_error = None

 for attempt in range(3):
 try:
 # Build prompt with error context
 if last_error:
 prompt = f"""
 Previous attempt failed with these errors:
 {format_validation_errors(last_error)}

 Please fix these specific issues and extract again.

 Text: {text}
 """
 else:
 prompt = f"Extract the following text according to the schema: {schema}"

 result = llm.extract(prompt, schema)
 return result
 except ValidationError as e:
 last_error = e

```

```

 """
 else:
 prompt = f"Extract information: {text}"

 result = llm.complete(prompt)
 return schema.parse_raw(result)

 except ValidationError as e:
 last_error = e

 # Analyze error and add specific instructions
 if has_type_error(e):
 # Add type hints for next attempt
 text = f"{text}\n\nIMPORTANT: Use correct types (numbers, strings, booleans, etc.)"

 if has_missing_required_field(e):
 missing = get_missing_fields(e)
 text = f"{text}\n\nIMPORTANT: Include these required fields: {missing}"

 raise ExtractionError(f"Failed after 3 attempts: {last_error}")

def format_validation_errors(error: ValidationError) -> str:
 """Convert Pydantic errors to human-readable feedback"""
 messages = []
 for err in error.errors():
 field = err['loc'][0]
 error_type = err['type']
 message = err['msg']

 messages.append(f"- Field '{field}': {message}")

 return "\n".join(messages)

```

**Impact:** 30-40% improvement in second/third attempt success rates.

**Advanced: Learn from errors to improve prompts globally**

python

```

class ErrorAnalyzer:
 def __init__(self):
 self.error_patterns = Counter()

 def record_error(self, schema_name: str, error: ValidationError):
 """Record error pattern"""

```

```

 for err in error.errors():
 pattern = (schema_name, err['loc'][0], err['type'])
 self.error_patterns[pattern] += 1

def get_common_issues(self, schema_name: str, top_n=5):
 """Get most common validation errors for schema"""
 schema_errors = [
 (pattern, count)
 for pattern, count in self.error_patterns.items()
 if pattern[0] == schema_name
]
 return sorted(schema_errors, key=lambda x: x[1], reverse=True)[:top_n]

def generate_preventive_instructions(self, schema_name: str) -> str:
 """Generate instructions based on historical errors"""
 common_issues = self.get_common_issues(schema_name)

 if not common_issues:
 return ""

 instructions = ["Based on common mistakes, please ensure:"]

 for (_, field, error_type), count in common_issues:
 if error_type == 'missing':
 instructions.append(f"- Always include field '{field}'")
 elif error_type == 'type_error':
 instructions.append(f"- Field '{field}' must be the correct type")
 elif error_type == 'value_error':
 instructions.append(f"- Field '{field}' must meet validation rules")

 return "\n".join(instructions)

Usage
analyzer = ErrorAnalyzer()

Track errors over time
for extraction_attempt in production_requests:
 try:
 result = extract(extraction_attempt.text, Task)
 except ValidationError as e:
 analyzer.record_error('Task', e)

Generate improved prompts
preventive_instructions = analyzer.generate_preventive_instructions('Task')

Most common issues:

```

```
- Missing field 'due_date' (342 times)
- Field 'priority' type error (198 times)
- Field 'assignee' validation error (87 times)

Add to system prompt
system_prompt = f"""
{BASE_INSTRUCTIONS}

{preventive_instructions}
"""
```

## Blind Spot 4: Function Calling != Perfect (Still Needs Validation)

**Common teaching:** "Function calling guarantees structured output."

**Reality:** Function calling guarantees valid *JSON*, not valid *data*.

**What function calling guarantees:**

-  Valid JSON syntax
-  Correct field names (matches function schema)
-  Correct types (string, number, boolean, array)

**What it DOESN'T guarantee:**

-  Valid email addresses
-  Dates in the past when future required
-  Positive numbers when specified
-  Non-empty strings
-  Cross-field constraints

**Example:**

python

```
Function schema
function_schema = {
 "name": "create_user",
 "parameters": {
 "type": "object",
 "properties": {
 "name": {"type": "string"},
 "email": {"type": "string"},
 "age": {"type": "number"}
 }
 }
}
```

```

 }
 }
}

LLM might generate
{
 "name": "", # ❌ Empty string (valid JSON, invalid data)
 "email": "not-an-email", # ❌ Invalid format
 "age": -5 # ❌ Negative age
}

```

**All three fields pass JSON schema validation but fail business logic validation.**

**Solution: Always add Pydantic validation layer**

python

```

class User(BaseModel):
 name: str = Field(..., min_length=1) # Non-empty
 email: EmailStr # Valid email format
 age: int = Field(..., ge=0, le=150) # Positive, reasonable

Even with function calling, validate
function_result = call_function(...)

try:
 user = User(**function_result) # ✅ Catches invalid data
except ValidationError as e:
 # Retry with error feedback
 pass

```

**Strong engineer pattern:**

```

LLM → Function Calling → Pydantic Validation → Application Logic
 (JSON validity) (Data validity)

```

## Blind Spot 5: Token Boundaries Break Structure

**Common teaching:** Not mentioned at all.

**Reality:** LLMs generate text token by token. At each token, they must "remember" the JSON structure context.

**Why this causes failures:**

python

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

Target JSON
json_str = '{"name":"Alice","age":30,"email":"alice@example.com"}'

Tokenization
tokens = [enc.decode([t]) for t in enc.encode(json_str)]
print(tokens)

Output (actual token boundaries):
['{', '"', 'name', ':", '"', 'Al', 'ice', '"', ',', '"', 'age', ':", '30', ',', '"', 'email', ':", '"', 'alice@example.com', '"', '}', '']

At each token, LLM must "know":
- Are we inside a string? (affects quote handling)
- Did we just finish a value? (need comma or close brace)
- Is this the last field? (no comma)
```

### Failure at token boundaries:

python

```
LLM generates tokens correctly up to...
'{"name":"Alice","age":30,'

Next token should be '"' (start of "email" key)
But LLM generates '}' (closes object early)

Result:
'{"name":"Alice","age":30,}' # Invalid JSON (trailing comma)
```

### Why constrained decoding helps:

python

```
At each token position, constrained decoder:
1. Parses JSON-so-far
2. Determines valid next tokens
3. Masks invalid tokens
4. Samples only from valid tokens
```



```
Position: '{"name":"Alice","age":30,'
Valid next tokens: ['"]' (must start next key)
Invalid next tokens: ['}', ',', numbers, letters, ...]
LLM can ONLY generate '"'

Guarantees valid JSON structure
```

Tradeoff table:

Method	Guarantees	Speed	Compatibility
Prompt only	None	Fast	All models
JSON mode	Valid JSON syntax	Fast	OpenAI, Some others
Function calling	Valid JSON + schema	Fast	OpenAI, Anthropic
Constrained decoding	Perfect structure	Slow (2-4x)	Local models only

When to use each:

- **Production API calls:** Function calling (best balance)
- **Batch processing:** JSON mode (cheaper, fast enough)
- **Critical extraction:** Constrained decoding (100% reliability needed)
- **Rapid prototyping:** Prompt only (iterate fast, fix later)

## How This Perspective Prevents Failures

Fragile System (Amateur):

- "Just ask for JSON" in prompt
- No validation layer
- Single extraction attempt
- Trusts LLM output blindly
- **Result:** 60-70% success rate, crashes on malformed data

Robust System (Engineer):

- Pydantic schemas with rich validation

- Multi-strategy extraction (function calling → JSON mode → fallback)
- Error feedback loops (retry with improvements)
- Partial extraction on failures
- Comprehensive testing across diverse inputs
- **Result:** 98%+ success rate, graceful degradation